Markdown structure and KV cache experiments

Ajit Pratap Singh LDLS Assignment 0, 16 October 2026

We tested whether the answer to a question about one section stays the same after another section is edited or removed. The input was five Markdown documents. Both main tests passed on all five. The city guide had a small difference in an additional test: the answer gained a full stop. The independent batched prompts also differed from the separate runs.

Scope and method

We used Markdown as our domain. The five sample documents are synthetic and contain 464 to 790 model tokens each, so all are below the assignment's 2,000-token limit. The assignment allows one domain or a pair of domains, so one Markdown parser is enough for this team's part.

The parser reads headings, paragraphs, lists and code blocks using markdown-it-py. It writes a JSON tree containing tokens, positions, seq_ids, node_type and parent. The token IDs come from the model's own llama.cpp tokenizer. Small sections are merged using a 40-token threshold, with at most 32 sequences by default. Each leaf unit gets a sequence ID. Its sequence includes the tokens on the path from the root to that unit. Sibling sections can reuse position numbers because they belong to different sequences.

The C++ driver reads the JSON and fills the KV cache using these positions and sequence IDs. The attention mask restricts the question to its sequence and earlier positions. Tokens from parent sections can be shared across sequences, which requires a unified KV cache. The code trace is in docs/llama_cpp_trace.md. It covers llama.h, the batched example and the mask construction.

Local setup

The run used WSL2 Ubuntu, an Intel Xeon E5-2620 v4 and eight CPU threads. The model was Llama-3.2-1B-Instruct Q4_K_M with greedy decoding. We rebuilt llama.cpp and the driver at commit 318a471 using the model already on the machine. CMake 4.2.3 and g++ 15.2.0 were available. All 28 unit tests passed, and the five parsed documents passed validation. The full commit and model checksum are in results/environment.txt.

Document results

For T1, we changed text in section B and asked the same question about A. The answer stayed the same on all five documents. The patched cache was also checked against a fresh prefill of the edited document. For T2, we removed unrelated section C. Again, the answer stayed the same, and the number of freed KV cells matched the prediction. As a control, editing A changed the answer to the new fact in every document. With full attention, the B distractor changed the answer for the research paper and city guide.

Document	Tokens	Depth	Leaf tokens median (range)	Markup	T1 / T2
research paper	790	5	63.5 (49-116)	6.1%	PASS / PASS
software readme	547	4	84 (40-129)	16.6%	PASS / PASS
lab protocol	565	5	77 (40-92)	12.6%	PASS / PASS
meeting notes	504	5	53.5 (40-107)	9.5%	PASS / PASS
city guide	464	4	52 (46-78)	9.5%	PASS / PASS

Leaf units are the merged subtrees that own individual sequences. Markup counts tokens whose non-whitespace bytes are entirely syntax. Full section-size statistics and sequence counts are in results/summary.md.

Runtime and KV usage

Prefill took about the same time in both modes: 4.95 s for structured attention and 4.92 s for flat attention. The difference showed up after editing B. Patching the affected sequences took 484 ms, while a full re-prefill took 5.00 s. Removing C took about 50 microseconds. These are medians across the five documents from one run. We did not measure variation over repeated trials. The extra logit tracing was disabled for these timings.

Removing C freed between 48 and 129 occupied KV cells. Each cell holds 32 KiB here, giving 1.5 to 4.0 MiB of space that the cache can reuse. This measures occupied cells; it does not show a drop in process RAM or the allocated cache buffer. Keeping only A's path left between 157 and 275 cells. That extra test passed on four documents and failed on the city guide.

Batched prompts

The two independent prompts did not give exactly the same outputs when run together and separately. They first differed at steps 3 and 24. We then changed the neighbouring prompt while keeping the batch shape the same. The observed sequence had zero logit difference in that check. For the shared-prefix case, both answers matched exactly, using 56 KV cells instead of 72. These checks help test sequence isolation, but the independent-prompt comparison still fails.

Teammate examples

The supplied HTML and LaTeX examples use one driver for both parsers. One version also combines five documents per domain into a corpus. Their JSON contains text segments and estimated positions. Our Markdown JSON contains actual model token IDs and explicit positions, so the files cannot be passed directly between the two drivers. They would need an adapter or an agreed schema. We read these examples but did not rerun them.

Why the city guide test failed

The city question asks how much a Maritime Museum ticket costs. The normal answer is "$12". With only A's path left in the cache, the answer becomes "$12.". The price is still correct, but the extra full stop makes the exact-token test fail. The remaining cache size is correct at 166 cells. The first two generated tokens match. At step 2, the normal run prefers stopping over a full stop by 0.208 logits. In the trimmed-cache run, the full stop wins by 0.036.

The model is close to a tie at this step, so a small change in the logits is enough to change the output. Floating-point differences after the cache layout changes are a possible cause. We have not isolated the exact operation responsible. The main T1 and T2 tests still pass for this document. We left the extra test marked as failed because it requires identical tokens.

Step 2 decision	Selected token	Runner-up	Margin
Baseline	end-of-turn (128009)	period (13)	0.208
Only A’s path	period (13)	end-of-turn (128009)	0.036

Limitations

The results cover five small synthetic documents. A question can read one root-to-leaf path, so a question needing information from two sibling sections would need a different arrangement. Prefill still processes every token, which explains why it was not faster. In this run, the useful difference was the lower cost of updating one section and the ability to reuse freed KV cells.

Running the experiment

Run the commands in README.md from the repository root after activating the Python virtual environment. The default document runner checks the required T1 and T2 tests. Add --strict to also enforce the optional keep-path comparison. The batched runner’s --strict option also requires independent prompts to match. The city logit trace is in results/diagnostics/05_city_guide.json. Environment versions and the model checksum are in results/environment.txt.

Credits

Original implementation: Rupinder Kaur and Abhishek, as credited in the supplied project. Submission and local reproduction: Ajit Pratap Singh. Earlier project notes acknowledge Claude (Anthropic). This update used Codex (OpenAI) for code fixes, experiment execution, diagnostics and editing. The project uses llama.cpp, markdown-it-py and Meta Llama 3.2, with the Q4_K_M GGUF from bartowski.